Lecture 22 (B21): Deep Surrogate Models

Pseudo-states, structural estimation, UQ, optimal policy, implied vol surfaces

Simon Scheidegger

HEC, University of Lausanne

Outline

Deep Surrogate Models

Motivation: Why Surrogates?

The computational bottleneck

Economic & financial models are expensive to evaluate:

- Solving Bellman equations / dynamic programming (DEQNs)
- PDE-based models (HJB, Black–Scholes for exotics)
- Monte Carlo simulations for risk measures

Key insight

We know the true model — we can generate *unlimited* training data by solving the model on a design of experiments.

Goal: Build a surrogate $\phi(\cdot \mid \theta_{NN})$ that approximates $f(\cdot)$ but evaluates *orders of magnitude faster*.

Look-up Tables $\rightarrow$ Neural Networks

Traditional: pre-compute & interpolate.

x	$\Phi(x)$
-2.0	0.0228
-1.0	0.1587
0.0	0.5000
1.0	0.8413
2.0	0.9772

Example: standard normal CDF table.

21st-century look-up table:

A neural network memorizes $\boldsymbol{x} \mapsto y$:

$$\phi(\boldsymbol{x}_t \mid \theta_{NN}) \approx y_t = f(\boldsymbol{x}_t)$$

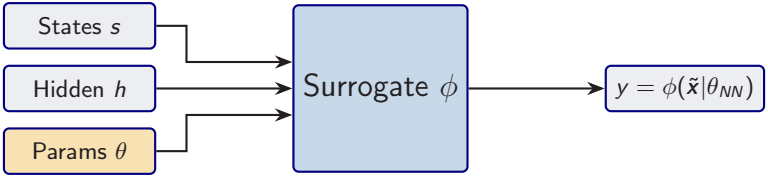
Continuous, differentiable, works in any dimension.

Pseudo-States: The Key Idea

Core insight (Chen, Didisheim & Scheidegger 2026)

Treat model parameters θ as extra state variables:

$$\tilde{\boldsymbol{x}} = \big(\underbrace{s_1, \ldots, s_n}_{\text{states}},\ \underbrace{h_1, \ldots, h_h}_{\text{hidden}},\ \underbrace{\theta_1, \ldots, \theta_p}_{\text{parameters}}\big) \in \mathbb{R}^d, \quad d = n + h + p.$$



Solve the model once over the full augmented space $\Rightarrow$ instant evaluation for *any* parameter configuration.

Building the Surrogate

1. **Design:** Draw $\tilde{\boldsymbol{x}}_i$ uniformly from $[\underline{x}, \bar{x}]^d, \quad i = 1, \ldots, m$.
2. **Label:** Compute $y_i = f(\tilde{\boldsymbol{x}}_i)$ using the expensive model.
3. **Train:** Minimize the empirical risk:

$$\hat{\theta}_{NN} = \arg\min_{\theta_{NN}} \frac{1}{m} \sum_{i=1}^{m} \left| \phi(\tilde{\boldsymbol{x}}_i \mid \theta_{NN}) - y_i \right|^2.$$

4. **Validate:** On a held-out test set, compute max / mean absolute error.

Remark

Since we can generate *unlimited* data from the true model, the training set can be made as large as needed $\Rightarrow$ no overfitting risk in the traditional sense.

Why Surrogates $\neq$ Standard ML

Standard supervised learning:

- Data is *scarce* and noisy
- Overfitting is a major concern
- Regularization, early stopping, cross-validation
- Small learning rates, careful tuning

Surrogate modeling:

- Data is unlimited (we own the model)
- Labels are exact (or noise-free)
- Can use large epochs, aggressive LR
- Focus on *approximation error*, not generalization

Once trained

- Highly accurate (errors $< 10^{-4}$ relative)
- Orders of magnitude faster than the original model
- Differentiable: gradients via automatic differentiation (backprop)

Speed Comparison: Option Pricing

From Chen, Didisheim & Scheidegger (2026), *J. Financial Economics*:

Task	**FFT**	**Surr. (CPU)**	**Surr. (GPU)**
Single price	23 ms	0.03 ms	—
1-day calib. (6×10^5)	3.3 h	18 s	0.4 s
1-year calib. (1.5×10^8)	$\approx$100 d	1.2 h	98 s

Takeaway

Deep surrogates enable real-time calibration of option pricing models — tasks that previously took days or were infeasible.

Comparison of Approximation Methods

Criterion	Polynomials/Splines	Sparse Grids	GPs	DNNs
High dimension ($d > 10$)	✗	∼	✗	✓
Local features / kinks	✗	✓	∼	✓
Irregular domains	✗	✗	✓	✓
Large training set ($N > 10^4$)	✓	∼	✗	✓
Built-in uncertainty	✗	✗	✓	✗

- **DNNs** excel in high dimensions with large data.
- **GPs** provide built-in uncertainty quantification — crucial for active learning and Bayesian inference.
- **Combining both** (GP for moderate d with UQ, DNN for high d) is a powerful strategy.

Application I: Structural Estimation

Goal: Estimate structural parameters θ from data via SMM:

$$\hat{\theta}_{SMM} = \arg\min_{\theta} \, [m(\theta) - \hat{m}^{\text{data}}]^\top \, W \, [m(\theta) - \hat{m}^{\text{data}}],$$

where $m(\theta)$ are model-implied moments and computing them naively requires solving and simulating the model for each candidate θ.

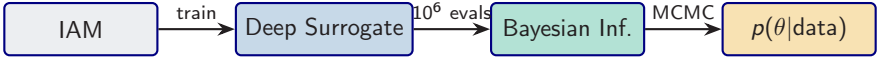
Surrogate-based estimation

1. Train a surrogate policy $\phi(x, \theta)$ over states and parameters (pseudo-states).
2. Reuse the trained surrogate to simulate moments $m(\theta)$ without re-solving the model.
3. Evaluate the SMM criterion over many θ values via grid search or standard optimization.

Result: Repeated SMM evaluations in seconds instead of weeks.
(Chen, Didisheim & Scheidegger 2026)

Application II: Uncertainty Quantification

DeepUQ (Friedl, Kübler, Scheidegger & Usui 2023):



- Surrogate over 100+ climate and economic parameters
- Posterior distributions of damage functions and optimal carbon tax
- MCMC requires millions of model evaluations, only feasible with a surrogate

Application III: Optimal Policy

Kübler, Scheidegger & Surbek (2025):

- High-dimensional integrated assessment model (IAM)
- GP surrogate + Bayesian Active Learning (BAL) for the value function
- GP provides *uncertainty quantification* at each point
- BAL selects training points where uncertainty is highest
- Result: optimal carbon tax policy with rigorous error bounds

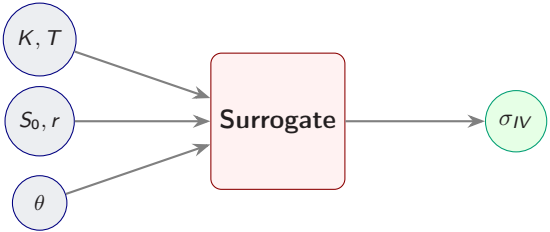
Preview

Parts III and IV of this lecture introduce Gaussian Processes and Bayesian Active Learning — the tools behind this application.

Application IV: The Implied Volatility Surface

Goal: Learn the mapping from option characteristics to implied volatility.

$$\sigma_{IV} = \phi(K, T, S_0, r, \theta_{\text{model}})$$



Benefits:

- **Calibration:** Invert the surrogate to find θ_{mod} matching market prices.
- **Arbitrage-free:** Can enforce constraints (no calendar/butterfly arb) in the loss.
- **Speed:** Enables real-time pricing for complex stochastic volatility models.

Pseudo-States: Concrete Use Cases

1. Exploring parameter distributions

Surrogate $\phi(s, \theta)$ maps *any* parameter θ to the model output instantly:

- Draw $\theta^{(j)} \sim p(\theta)$ from a prior or posterior distribution
- Evaluate $\phi(s, \theta^{(j)})$ for each draw — no re-solving
- Obtain the full distribution of outcomes: $\{y^{(j)}\}_{j=1}^{M}$

$\Rightarrow$ Bayesian inference, sensitivity analysis, robust policy design

Example: Climate–economy models

- Parameters: climate sensitivity, damage curvature, discount rate
- Surrogate over 100+ parameters trained once ($\sim$hours)
- MCMC posterior with 10^6 evaluations ($\sim$minutes with surrogate)

Pseudo-States: Policy Parameterization

2. Parameterizing policies

Include policy parameters θ_{pol} as pseudo-states. Optimize via autograd:

$$\theta^*_{\text{pol}} = \arg\max_{\theta_{\text{pol}}} \phi\big(s, \theta_{\text{model}}, \theta_{\text{pol}}\big)$$

Carbon tax

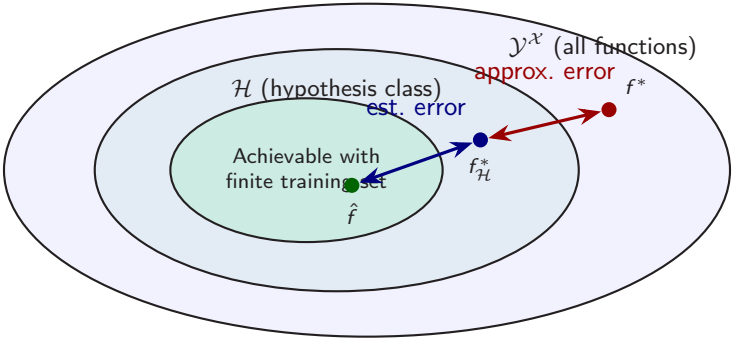
- Policy: tax trajectory θ_{pol}
- Model: climate sensitivity θ_{model}
- Optimal tax for each θ_{model}

Monetary policy

- Policy: Taylor rule coefficients
- Model: preference / technology shocks
- Optimal rule via gradient descent

Key: evaluate welfare for *any* model + policy configuration instantly $\Rightarrow$ no re-solving.

Sources of Approximation Error



- **Approximation error:** f^* may not lie in $\mathcal{H}$ $\Rightarrow$ increase network capacity.
- **Estimation error:** finite data $\Rightarrow$ mitigated by generating more training points.
- For surrogates, estimation error $\rightarrow 0$ as we can always generate more data.

Practical Tips for Surrogate Training

Architecture

- Swish activation ($x \cdot \sigma(x)$)
- 3–5 hidden layers, 128–512 neurons
- Linear output (no activation)

Validation

- Held-out test set
- Max / mean absolute error, R^2
- Scatter: prediction vs. truth

Training

- $m \geq 10^5$ samples (cheap to generate)
- Adam, LR 10^{-3}, reduce on plateau
- Hundreds of epochs (no overfit risk)

Loss functions

- MSE: $\frac{1}{m}\sum_i(\phi_i - y_i)^2$
- Relative L^1: $\frac{1}{m}\sum_i|(\phi_i - y_i)/y_i|$

Summary: Deep Surrogate Models

1. **Surrogates** = cheap, accurate replicas of expensive models.
2. **Pseudo-states** (θ as extra inputs) unlock:
   - Structural estimation (GMM in seconds)
   - Uncertainty quantification (MCMC with surrogate)
   - Optimal policy design
3. **Key advantage:** unlimited training data $\Rightarrow$ no overfitting.
4. **Speed:** orders of magnitude faster than original model.

Hands-on

Notebook: `01_Surrogate_Primer.ipynb` — Black–Scholes surrogate with implied volatility inversion.